RRC Bus Servo Simple Rotation and Source Code Analysis

1. Servo Description

It's an automatic control system consisting of a DC motor, reduction gear set, sensor, and control circuitry, which specifies the rotation angle of the output shaft by sending signals.

Differences between a servo and a motor:

Servos generally have a maximum rotation angle; DC motors rotate in full circles. Therefore, servos can only rotate within a certain angle and cannot rotate continuously like DC motors. Regular DC motors cannot provide feedback on their rotation angle, while servos can. (Digital servos can switch between servo mode and motor mode.)

2. Serial Bus Servo

Regular DC motors typically rotate in full circles to provide power, while servos are used to control the rotation of an object to a specific angle.

Bus servos are derivatives of digital servos that communicate via an asynchronous serial bus, meaning control is achieved through the sending and receiving of instruction packets, representing a closed-loop control system. Due to this feature, bus servos can be daisy-chained, simplifying wiring and reducing serial port usage. Consequently, when connecting bus servos, it's necessary to set the ID for each servo beforehand; otherwise, it would be impossible to differentiate between them during control.

Bus servos internally utilize high-precision potentiometers. Compared to PWM

servo control, they not only have the ability to provide feedback on position, temperature, voltage, and other information, but also exhibit excellent accuracy and linearity. This makes robot operation more stable and significantly extends the lifespan of servos.

Additionally, before assembling servos, it's essential to perform a midpoint operation. The midpoint of the servo refers to its initial position, which serves as the "zero point" for positive and negative angle rotations. Therefore, servos should be adjusted to the midpoint before installation onto servo horns.

This is because, during servo rotation, the moving parts drive the potentiometer. The software assumes the midpoint as the "zero point"; otherwise, the potentiometer might enter a "blind spot," causing the entire component to malfunction. Consequently, when using the robot, the specified angles may not be achieved, or the corresponding action sets may not match.

3. Program Outcome

We rotated the servo counterclockwise by 90 degrees, then returned it to 0 degrees, and finally stopped the servo.

4. Code Analysis

File location: Hiwonder/app.c. This file implements the simple angle rotation of the servo and sets the position and duration of the servo's movement.

```
/* 用户入口函数 */
void app_task_entry(void *argument)
{

    /* 硬件初始化 */
    serial_servo_set_position(1, HOLDER_MIN, 1000); //将舵机转到位置500，并运动时间为1000毫秒
    //serial_servo_set_position(3, HOLDER_MAX, 1000);//将舵机转到位置2500，并运动时间为1000毫秒
    osDelay(1000);
    serial_servo_set_position(1, 0, 1000);
    serial_servo_set_position(3, 0, 1000);


    osDelay(1500);//程序.5秒后停止
//  printf("Current servo ID is: %d\n", serial_servo_set_id);
    serial_servo_stop(1);//
```

The function serial_servo_set_position() is used to set the position and duration of movement for the servo. Specific position and duration information is sent to the servo via a serial data frame (defined by the array buffer[]). The execution process of the function is as follows:

1) Define a buffer to store the sent data frame.
2) If the target position is greater than 1000, it is set to 1000.
3) Fill the content of the data frame: frame header, servo ID, buffer[3] setting data length, set the position and movement duration of the servo.
4) Store the low byte and high byte of the target position in buffer[5] and buffer[6], and store the low byte and high byte of the movement duration in buffer[7] and buffer[8]. Calculate the checksum, and finally send the data frame via the serial port.

```
void serial_servo_set_position(uint8_t servo_id, uint16_t position, uint16_t duration)
    if (servo_id > 31) {  //舵机ID不能大于31,可根据对应控制板修改
        return;
    }
    uint8_t buffer[10];
    position = position > 1000 ? 1000 : position;
    buffer[0] = SERIAL_SERVO_FRAME_HEADER;
    buffer[1] = SERIAL_SERVO_FRAME_HEADER;
    buffer[2] = servo_id;
    buffer[3] = 7;
    buffer[4] = SERIAL_SERVO_MOVE_TIME_WRITE;
    buffer[5] = GET_LOW_BYTE(position);
    buffer[6] = GET_HIGH_BYTE(position);
    buffer[7] = GET_LOW_BYTE(duration);
    buffer[8] = GET_HIGH_BYTE(duration);
    buffer[9] = CheckSum(buffer);
    SerialWrite(buffer, 10);
}
```

The serial_servo_stop() function is designed to send a command to stop the

servo via the serial port. The execution process is as follows:

1) Define a buffer to store the sent data frame.
2) Fill the content of the data frame: frame header, servo ID, data length, stop the servo, calculate the checksum, and finally send the data frame via the serial port.

```
118 void serial_servo_stop(uint8_t servo_id) {
119     uint8_t buffer[10];
120     buffer[0] = SERIAL_SERVO_FRAME_HEADER;
121     buffer[1] = SERIAL_SERVO_FRAME_HEADER;
122     buffer[2] = servo_id;
123     buffer[3] = 6;
124     buffer[4] = SERIAL_SERVO_MOVE_STOP;
125     buffer[5] = CheckSum(buffer);
126     SerialWrite(buffer, 6);
127 }
```